JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time.format

Enum SignStyle

java.lang.Object
    java.lang.Enum<SignStyle>
        java.time.format.SignStyle

All Implemented Interfaces:

Serializable, Comparable<SignStyle>

```
public enum SignStyle
extends Enum<SignStyle>
```

Enumeration of ways to handle the positive/negative sign.

The formatting engine allows the positive and negative signs of numbers to be controlled using this enum. See DateTimeFormatterBuilder for usage.

Implementation Requirements:

This is an immutable and thread-safe enum.

Since:

1.8

Enum Constant Summary

Enum Constants

Enum Constant	Description
ALWAYS	Style to always output the sign, where zero will output '+'.
EXCEEDS_PAD	Style to always output the sign if the value exceeds the pad width.
NEVER	

	Style to never output sign, only outputting the absolute value.
NORMAL	Style to output the sign only if the value is negative.
NOT_NEGATIVE	Style to block negative values, throwing an exception on printing.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`static SignStyle`	**`valueOf`**`(String name)`	Returns the enum constant of this type with the specified name.
`static SignStyle[]`	**`values`**`()`	Returns an array containing the constants of this enum type, in the order they are declared.

Methods inherited from class java.lang.Enum

`clone`, `compareTo`, `equals`, `finalize`, `getDeclaringClass`, `hashCode`, `name`, `ordinal`, `toString`, `valueOf`

Methods inherited from class java.lang.Object

`getClass`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Enum Constant Detail

NORMAL

```
public static final SignStyle NORMAL
```

Style to output the sign only if the value is negative.

In strict parsing, the negative sign will be accepted and the positive sign rejected. In lenient parsing, any sign will be accepted.

ALWAYS

```
public static final SignStyle ALWAYS
```

Style to always output the sign, where zero will output '+'.

In strict parsing, the absence of a sign will be rejected. In lenient parsing, any sign will be accepted, with the absence of a sign treated as a positive number.

NEVER

```
public static final SignStyle NEVER
```

Style to never output sign, only outputting the absolute value.

In strict parsing, any sign will be rejected. In lenient parsing, any sign will be accepted unless the width is fixed.

NOT_NEGATIVE

```
public static final SignStyle NOT_NEGATIVE
```

Style to block negative values, throwing an exception on printing.

In strict parsing, any sign will be rejected. In lenient parsing, any sign will be accepted unless the width is fixed.

EXCEEDS_PAD

```
public static final SignStyle EXCEEDS_PAD
```

Style to always output the sign if the value exceeds the pad width. A negative value will always output the '-' sign.

In strict parsing, the sign will be rejected unless the pad width is exceeded. In lenient parsing, any sign will be accepted, with the absence of a sign treated as a positive number.

Method Detail

values

```
public static SignStyle[] values()
```

Returns an array containing the constants of this enum type, in the order they are declared. This method may be used to iterate over the constants as follows:

```
for (SignStyle c : SignStyle.values())
    System.out.println(c);
```

Returns:

an array containing the constants of this enum type, in the order they are declared

valueOf

```
public static SignStyle valueOf(String name)
```

Returns the enum constant of this type with the specified name. The string must match *exactly* an identifier used to declare an enum constant in this type. (Extraneous whitespace characters are not permitted.)

Parameters:

name - the name of the enum constant to be returned.

Returns:

the enum constant with the specified name

Throws:

IllegalArgumentException - if this enum type has no constant with the specified name

NullPointerException - if the argument is null

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.